SMITH ENGINEERING

Queen's University

Department of Electrical and Computer Engineering



ELEC 490/498 Project Proposal

Multi-Layer Satellite Simulator for UAV Detection

Fall 2026 / Winter 2027

Submitted by Group 9

Group member	NetID	Student number
Mihailo Ratkov	22fq22	20395168
Ben Henning	22vgr2	20394046
Luka Kourkopoulos	23drvq	20423023

Instructor: Heidi Miller

TA: Dimitria Silveria

Faculty supervisor(s): Scott Yam

Supervisor email: scott.yam@queensu.ca

Submission date: 07 October 2026

Version: 1

Executive Summary

In the contemporary era, Low Earth Orbit (LEO) satellites are becoming the most common tool for aerial target detection from space; the U.S. Space Force and its PWSA and SB-AMTI are evidence of this. However, there is currently minimal research that combines LEO with Medium Earth Orbit (MEO) and Geostationary Earth Orbit (GEO) satellites in these remote sensing systems, despite their unique traits. Herein lies the main problem to be solved, an aerial target detection multi-layer satellite system (MLSS), as well as a method to test these theories.

Towards this goal, a rigorous simulation system is to be designed that will allow for user-defined testing in a physics-accurate environment. It will support all three layers of satellites around a model of the Earth, with user-configurable satellite models, constellations, and aerial targets, allowing for a fully customizable testing environment.

The level to which realistic conditions will be implemented is subject heavily to scope. Real-time simulation and physics-based infrared sensors and networks were intentionally not included in the objectives out of concern for computational and time constraints.

The system will be delivered with four main components, each of which will be first delivered as prototypes in successive order, as follows: the C++ physics engine, an environment model that handles the satellite networks and sensors, a PySide6 graphical user interface, and a performance monitor to evaluate key metrics such as coverage and accuracy of aerial target tracking. These will be tested with an iterative series of unit verification tests, before being combined in an integration validation test and packaged as the final prototype. All resources used will be free software, like NumPy, PySide6, and OpenSky Vector Data, resulting in no expected cost.

Contents

Executive Summary i

Acronyms v

1 Introduction 1

1.1 Problem and Purpose . . . 1

1.2 Project Objective . . . 1

1.3 Scope and Deliverables . . . 1

2 Functional Description 2

2.1 System overview . . . 2

2.2 Interface and performance specifications . . . 4

2.3 Risks and safety hazards . . . 5

3 Design and Production Approach 5

3.1 Development Process . . . 5

3.2 Milestones and Division of Labor . . . 5

3.3 Contingencies . . . 6

4 Testing and Evaluation 7

5 Resource Requirements 7

5.1 Access to Resources . . . 8

5.2 Resource Costs . . . 8

6 Conclusion 8

7 References 9

A Supporting Schedule and Technical Material 10

List of Figures

1 High level system diagram of the multi-layer satellite simulator, illustrating the general workflow and the types of inputs/outputs the multi-layer satellite simulator (MLSS) will be responsible for 2
2 Subsystem diagram of the proposed design of the MLSS 3
3 State machine diagram of the MLSS 4
4 GUI and Backend Controls joint functional block diagram (FBD) 10
5 Satellite Object and Environment Model joint FBD 11
6 Run Data Store and Performance Monitor joint FBD 12
7 Physics Engine FBD 13
8 Sequence diagram for scenario configuration and validation 14
9 Sequence diagram for scenario pre-computing simulation run 15
10 Sequence diagram for analyzing simulation run 16
11 Sequence diagram for user playback of simulation run 17
12 Full testing and prototype process diagram 18

List of Tables

1 Preliminary system specifications. 4
2 Technical milestones, deadlines, and ownership. 5
3 Brief description of tests for each requirement and the metrics of success. . 7
4 Resource access by milestone. 8

Acronyms

AMTI	Airborne Moving Target Indicator
GEO	geostationary
GUI	graphical user interface
LEO	low earth orbit
MEO	medium earth orbit
MLSS	multi-layer satellite simulator
TCP	transmission control protocol
UAV	unmanned aerial vehicles
FBD	functional block diagram
MVP	minimal viable product
GPUs	graphical processing units
GUI	graphical user interface

1. Introduction

1.1 Problem and Purpose

Over the past few years, unmanned aerial vehicles (UAV) have become widely adopted for numerous applications such as search and rescue, precision agriculture, and wireless networks, among others. Understandably, concerns regarding monitoring the these vehicles in airspace have arisen [1]. Simultaneously, satellite constellations have been rapidly proliferating, especially in the low earth orbit (LEO) and medium earth orbit (MEO) layers, with small target tracking capabilities becoming more reliable and efficient. [2].

Considering the need for precise tracking of UAVs, it makes sense to surveil them with infrared satellites at the LEO, MEO, and geostationary (GEO) layers. This is known as the Airborne Moving Target Indicator (AMTI) problem. Current literature on this topic explores aerial object monitoring using intra-layer and inter-layer communication between pairs of layers (i.e. LEO and MEO) [3], [4]. However, there is little research on incorporating all three layers to solve AMTI. This work seeks to fill that gap.

1.2 Project Objective

A simulator will be developed to model satellite orbit, communication, sensing, and target tracking. Simulations will be able to model single and multi-layer constellations, allowing for comparisons between established satellite configurations like the Walker and a custom built configuration communicating between all three layers. A high-level system diagram of the simulator is provided in Figure 1.

The modeling approach for satellite trajectories follows related works like Jamal et al., using Kepler's equation to model simple two-body orbits [3]. Kassing et al. delivers a LEO constellation simulator and utilizing Python frameworks (satgenpy and ns3-sat-sim) to model satellite movement and transmission control protocol (TCP) communication [5]. By contrast, this work will use Python numerical libraries like NumPy, Astropy, and ephem to simulate packet-level communications.

1.3 Scope and Deliverables

The MLSS will comprise four major components. Component interactions are outlined in Section 2. First, the simulator will feature a C++ physics engine to handle satellite orbit. The engine will implement a J2 propagator and simplify the problem by assuming

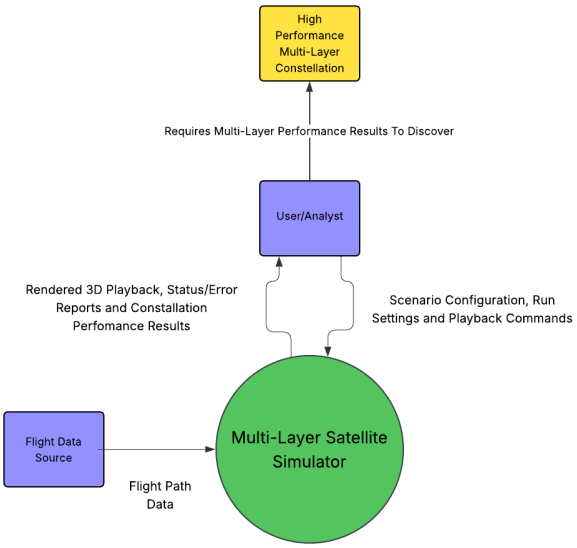


Figure 1: High level system diagram of the multi-layer satellite simulator, illustrating the general workflow and the types of inputs/outputs the MLSS will be responsible for

satellites will automatically correct for external gravitational forces from miscellaneous sources (moon/sun). Second, an environment model will be written in Python to handle satellite networking and sensor readings. This model will receive an ephemeris calculated from the physics engine and use that to make position calculations, simulate TCP packet transmissions, and confirm IR sensor readings. Third, a PySide6 graphical user interface (GUI) will present performance monitor and show the satellites and UAVs in space. Finally, the performance monitor will evaluate whether the input user satellite configuration will feasibly detect UAVs and communicate readings within and between satellite layers. This approach lacks real-time simulation which was an intentional design decision to reduce the scope, focusing effort on primarily delivering a functional product.

2. Functional Description

2.1 System overview

Figure 2 shows the proposed interactions between the subsystems; it can be seen how Environment Model is the only system with which Satellite Objects will be able to interact. It will add noise to their sensor readings and gate messaging behind a calculated link state. It is worth noting that Satellite Objects is being represented as one system, but in practice there will be many satellites interfacing with Environment Model to simulate inter-satellite communication.

The core of the MLSS is the subsystem titled Backend Controls as it takes user controls and the user inputted run configuration from the GUI to coordinate the run between the Physics Engine, Environment Model, Run Data Store and Performance Monitor subsystems. The GUI will also allow the user to interface with Run Data Store to replay the precomputed simulation in a fast 3D view in which performance metrics and simulation state (positions & links) can be extracted.

Due to the abundant need for data storage, the Run Data Store subsystem has been considered early. Its main job is to store all metrics of a run such that the user can view metrics and a playback of any run at any time in the simulation. These metrics are computed in the Performance Monitor after the Run Data Store relays the relevant run data to analyze.

The Physics Engine handles the production of the positions of every entity (satellite bodies, aircraft and ground stations) at every time, along with relevant error detection. Satellite bodies will be instantiated with their Kepler orbital elements used for kinematics. The engine will also instantiate aircraft and ground stations, given by the flight path data and configuration.

Higher technical depth on all the subsystems can be seen in Figures 4, 5, 6 and 7 in Appendix A

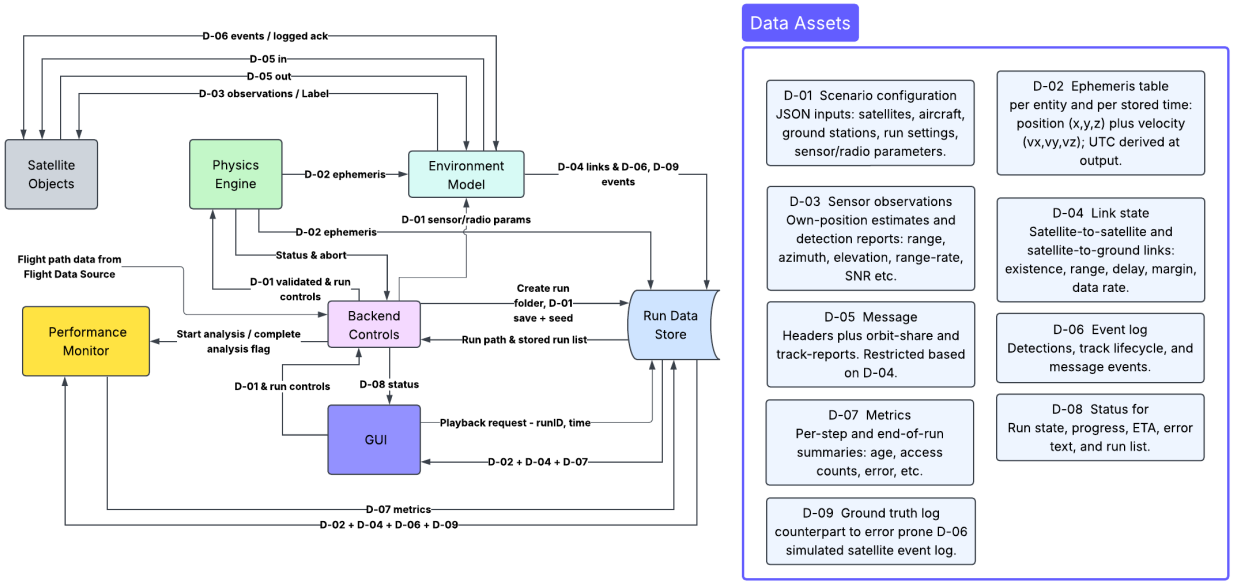


Figure 2: Subsystem diagram of the proposed design of the MLSS

The general usage flow of the MLSS is described as a state machine in Figure 3. Higher depth of the sequencing is represented as four phases of a sequence diagram seen in Figure 8, 9, 10 and 11 within Appendix A.

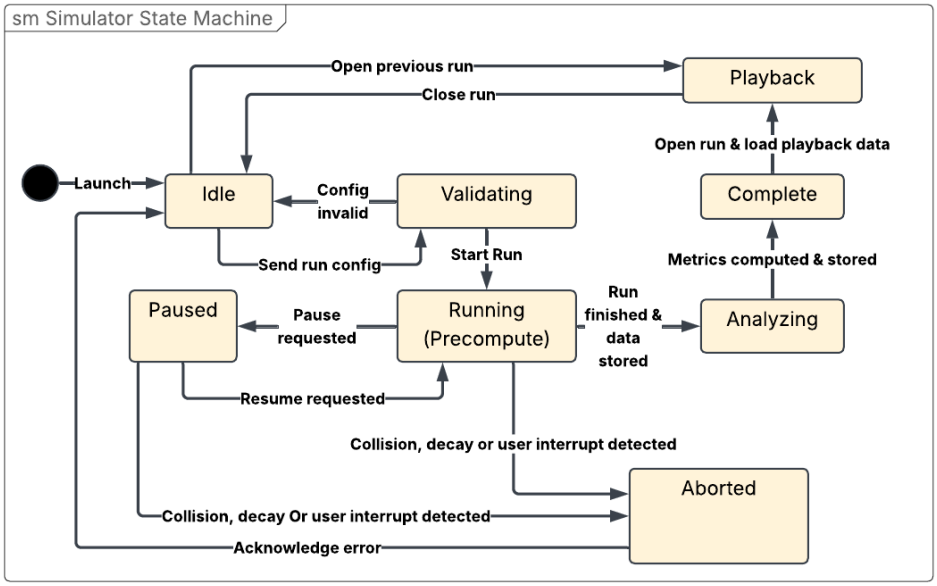


Figure 3: State machine diagram of the MLSS

2.2 Interface and performance specifications

Table 1 outlines the expectations of the functional design, not of the constellations being simulated. This is why the requirements are mostly in the form of accuracies of the simulator outputs. D-08 is given no tolerance because the system will always fail if the system is unsure what state its in. G-01 was added symbolically to show that this design requires the capability to produce a functioning multi-layer configuration.

Table 1: Preliminary system specifications.

ID	Requirement	Target	Tolerance
D-02a	Entity position	N/A, m	±1km
D-02b	Entity velocity	N/A, m/s at time	±1m/s
D-04a	Link existence accuracy	always correct	-5% accuracy
D-07	Constellation evaluation metrics	N/A	-5% accuracy
D-08	Run State Status	always correct	no tolerance
D-09	Ground truth event log	all events recorded	1% loss

ID	Requirement	Target	Tolerance
G-01	Multi-layer configuration	N/A	Performance can vary but must be functional

2.3 Risks and safety hazards

Since this project is entirely software based for research applications, there is no relevant health or safety risks worth considering.

Technical risks are abundant for this scope, and have been considered early. They can be reduced into accuracy risk, performance risk and feasibility risk. Accuracy risk is the concern that the simulator will not be capable of modeling reality to a high enough degree for the user's findings to be useful. The mitigation strategy for this is to rapidly prototype the satellites, Environment Model and Physics Engine to continue to improve realism. Conversely, while improving realism, performance will be reduced; mitigation from this comes from proactive performance analysis and plans to utilize higher compute with already owned local graphical processing units (GPUs). Feasibility risk comes from the general depth of this project. The mitigation strategy for feasibility risk is to start off with prototyping the minimal viable product (MVP) and expanding depth from there.

3. Design and Production Approach

3.1 Development Process

A heavily iterative approach will be followed for prototype development. This is reflected in the milestones, as each milestone builds on previous ones. For instance, prototype 2 will include the two-body propagator from prototype 1.

3.2 Milestones and Division of Labor

Table 2: Technical milestones, deadlines, and ownership.

ID	Milestone	Due date	Lead
M1	Prototype 1: Physics Engine (Two-Body Propagator), Backend Controls (Command Interface, Config Validator)	October 30th, 2026	Mihailo

M2	Prototype 2: Satellite Object, Environment Model (Link Model Only), Simple Performance Monitor	November 20th, 2026	Luka
M3	Prototype 3: GUI & Full Backend Controls	December 18th, 2026	Ben
M4	Prototype 4: Complete Environment Model, J2 Perturbation	January 15th, 2027	Luka
M5	Prototype 5: Complete Performance Monitor & Run Data Store	February 5th, 2027	Ben
M6	Final Prototype: Unit Testing, Debugging, Refining Simulator	March 15th, 2027	All Members

M1: Implements two-body orbits (without J2) and backend control, which receives configurations from files. Satellite coordinates saved to directory.

M2: Satellites are implemented as objects with sensor and TCP routing capabilities. The environment model only has link model and ground station handler to validate satellite configurations. Performance monitor only joins logs with D-02 and D-04 (see Figure 6.

M3: GUI implements scenario editing, 3D view, playback controller. Backend controls are upgraded to interface with GUI.

M4: Environment model now accounts for UAV ground truth data and models visibility and noise. Propagator is updated to include J2 perturbation.

M5: Run data store is set up and completed performance monitor is added to GUI, along with the incorporation of D-06 and D-09 information in logs.

M6: Testing should include unit tests for each subsystem. Potential simulator refinement includes, but is not limited to GUI enhancements, expansion of propagator options, and real-time functionality.

3.3 Contingencies

Time has been allocated to account for familiarity to be acquired with programming languages and GUI development frameworks. Features that aren't prerequisites for other features can be dropped if time runs out during prototype development. For example, a J2 propagator does not need to be implemented before moving on to prototype 5 as no subsequent prototype depends on J2 propagation.

4. Testing and Evaluation

The testing is to be conducted in an iterative manner, as seen in Appendix A in Figure 12. The tests are listed in Table 3 according to the requirement they are intended to satisfy and the expected evidence of success. Where applicable, tests are described as either verification or validation, denoting whether the goal is to prove correctness or to prove alignment with the stakeholder's goals.

Since it is a purely software-based approach, the inputs of each test are a series of either user-defined or randomized values to determine the model's reaction. Testing success will be measured by the results variance when compared to expected values.

Table 3: Brief description of tests for each requirement and the metrics of success.

ReqID	Proposed test	Expected evidence of success
D-02a	Mass entity placement verification tests, one planned and one randomized	Numerically ensuring entities can generate within expected bounds and are not colliding
D-02b	Mass entity movement verification tests, one planned and one randomized	Numerically ensuring no entity trajectories are outside their orbit, and are not colliding with each other or with Earth
D-04a	Line-of-sight and visibility verification tests	Satellites can only communicate if within line-of-sight, and they are limited by realistic conditions in their visibility
D-07	Unit verification test of constellation network communication	Successful transmission in a Walker-delta communication test between distant satellites, confirmed by event log
D-08	LEO tracking of single-target, integration with physics model validation test	Both visual confirmation and event log confirmation of tracking and handoffs
D-09	Localized single-region single-target tracking verification test	Success determined by accurate event log of involved satellites
G-01	Final integration with multi-layer tracking for multiple targets, validation test	Event log confirming inter-satellite communication, inter-layer communication (if within scope), accurate tracking of targets on ground

5. Resource Requirements

5.1 Access to Resources

Technical milestones rely on C++ and Python for implementation. External libraries and datasets required for simulator development are listed in Table 4. Access to these resources is free of charge.

Table 4: Resource access by milestone.

Milestone	Resource / source	Access date / lead time
M2, M4	NumPy, Astropy, ephem	October 30th, 2026
M3, M5, M6	GUI Tools (pybind11, PySide6, VTK)	November 20th, 2026
M4, M6	OpenSky Vector Data	December 18th, 2026

Numerical packages for M2 and M4 will be used to model constellations, parse and create data assets (D-01, D-02, D-03, D-04, D-07), and validate configurations. For M4 and M6, flight data (D-09) will be incorporated through a CSV file.

5.2 Resource Costs

The simulator will not require any financial investment since no paid APIs or cloud services are used for project implementation. Every subsystem can be completed through free libraries. Past flight data is also publicly available at no cost on OpenSky Network [6].

6. Conclusion

As LEO satellite constellations are becoming the state of the art for UAV detection, there is increasing need to refine this new technology to save costs and minimize its weaknesses. Research on multi-layer systems is one front to address this issue, but the lack of evaluation available with these systems makes research sparse and unverifiable. The MLSS is the design proposed to address this issue. MLSS allows performance of multi-layer systems to be accurately evaluated using a physics engine, a model for a satellite communication network, internal performance monitoring and a user friendly GUI. The general design of the MLSS was made to be rapidly prototype-able and extendable such that realism could continue to improve as need for multi-layer grows along-side it.

7. References

[1] Z. Tang, H. Ma, Y. Qu, and X. Mao, “Uav detection with passive radar: Algorithms, applications, and challenges,” *Drones*, vol. 9, no. 1, 2025, ISSN: 2504-446X. DOI: 10.3390/drones9010076 [Online]. Available: https://www.mdpi.com/2504-446X/9/1/76

[2] K. Underwood, “Moving up to meo,” *SIGNAL Media*, 2026. [Online]. Available: https://www.afcea.org/signal-media/moving-meo

[3] M. Jamal, E. Cianca, T. Rossi, and M. De Sanctis, “Multilayer satellite constellation design for multipurpose applications,” *Engineering Proceedings*, vol. 133, no. 1, 2026, ISSN: 2673-4591. DOI: 10.3390/engproc2026133177 [Online]. Available: https://www.mdpi.com/2673-4591/133/1/177

[4] Z. Kassas, “Navigation from low-earth orbit: Part 2: Models, implementation, and performance,” in Dec. 2020, pp. 1381–1412, ISBN: 9781119458494. DOI: 10.1002/9781119458555.ch43b

[5] S. Kassing, D. Bhattacherjee, A. B. Águas, J. E. Saethre, and A. Singla, “Exploring the “Internet from space” with Hypatia,” in *ACM IMC*, 2020.

[6] OpenSky. “Weekly 24 hours of state vector data 2017 - 2022,” Accessed: Oct. 6, 2026. [Online]. Available: https://opensky-network.org/data/scientific

A. Supporting Schedule and Technical Material

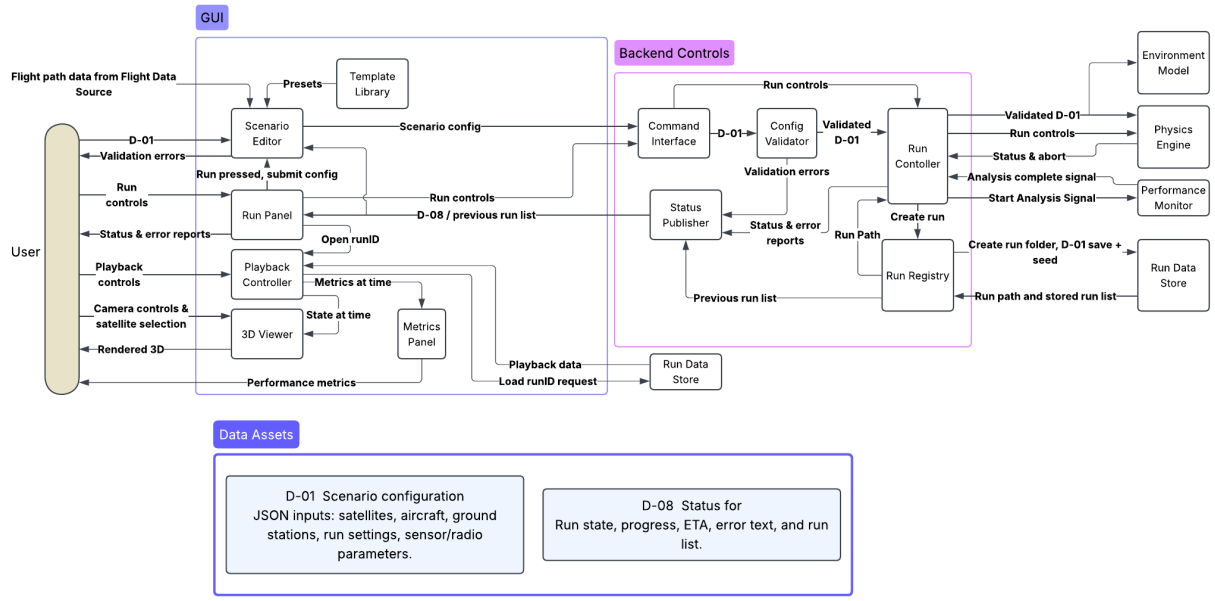


Figure 4: GUI and Backend Controls joint FBD

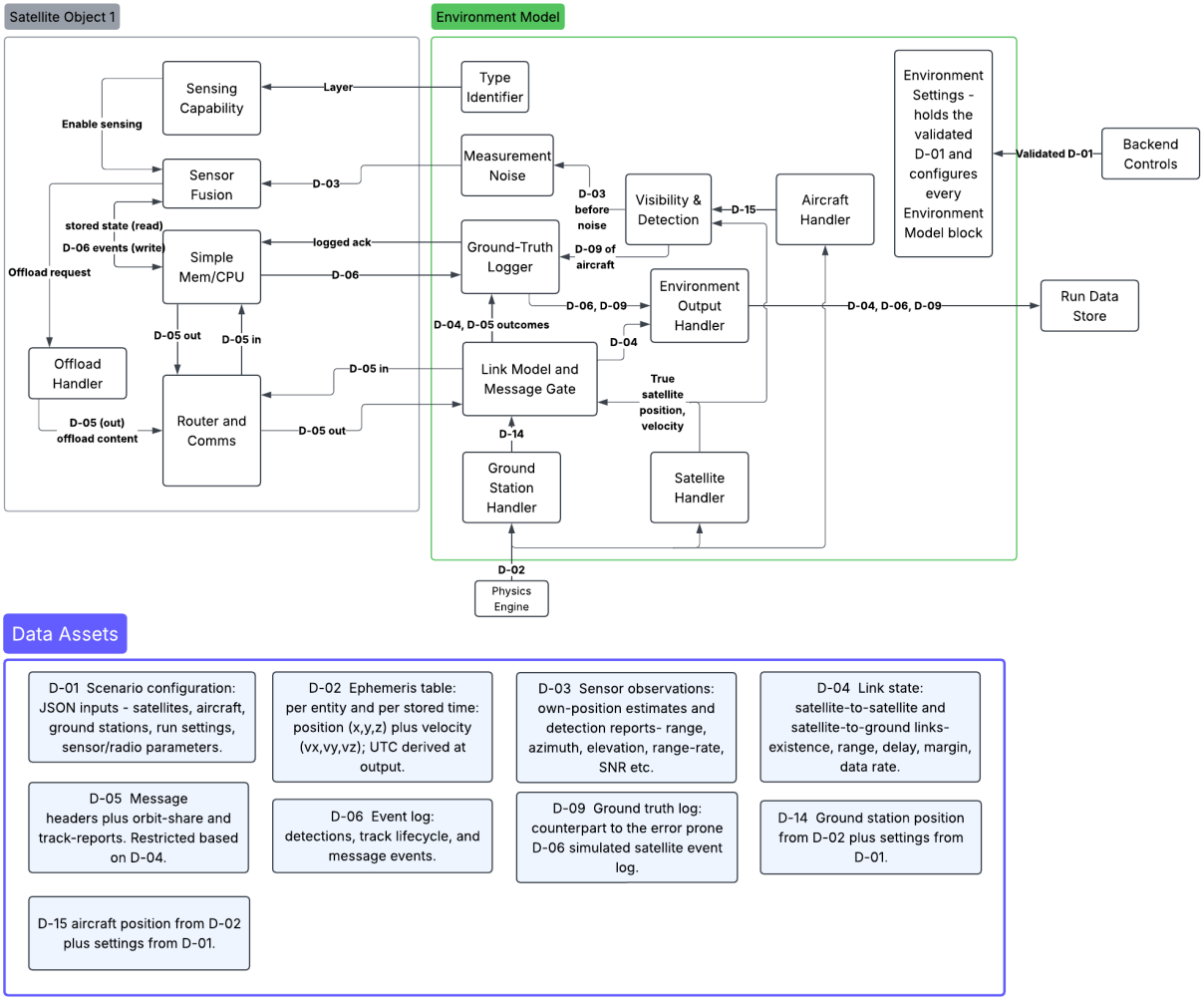


Figure 5: Satellite Object and Environment Model joint FBD

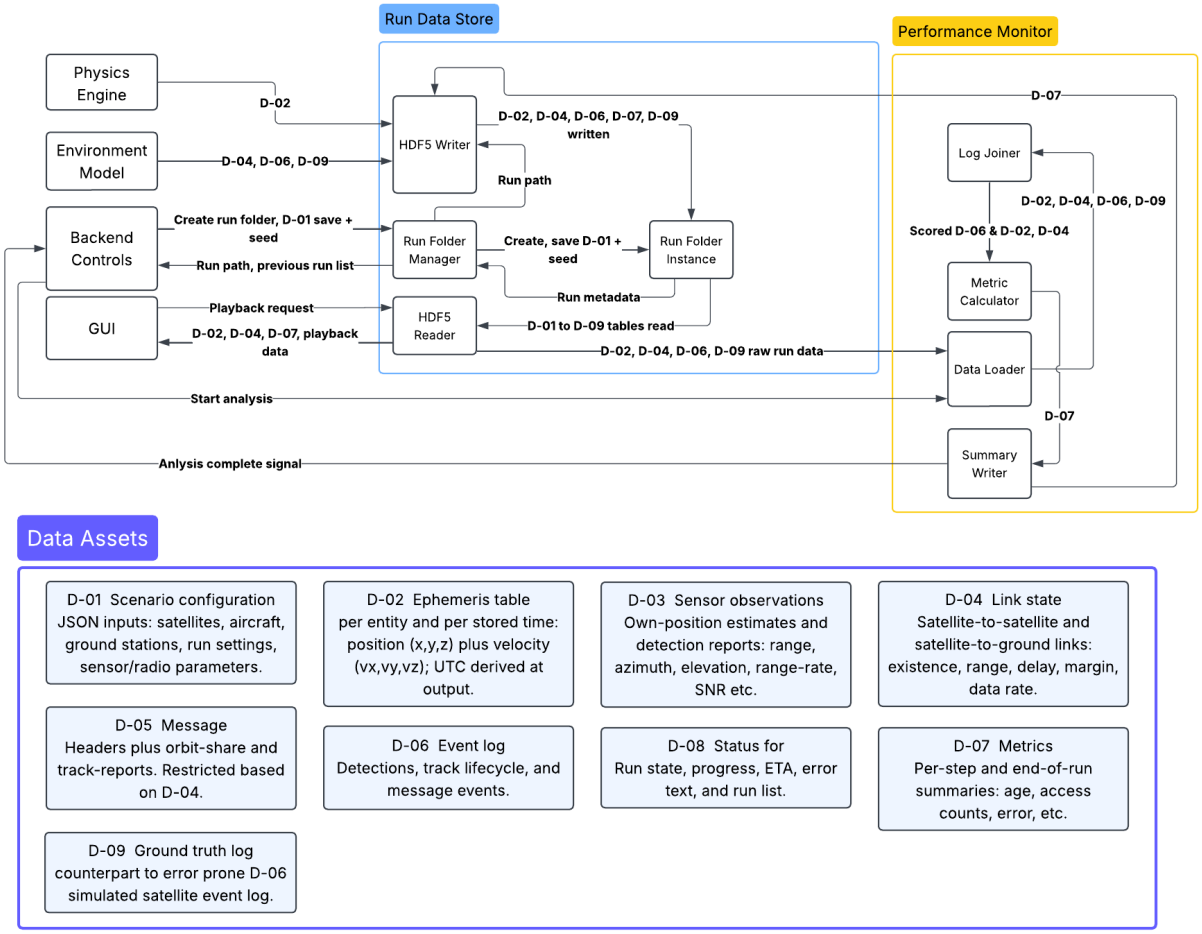


Figure 6: Run Data Store and Performance Monitor joint FBD

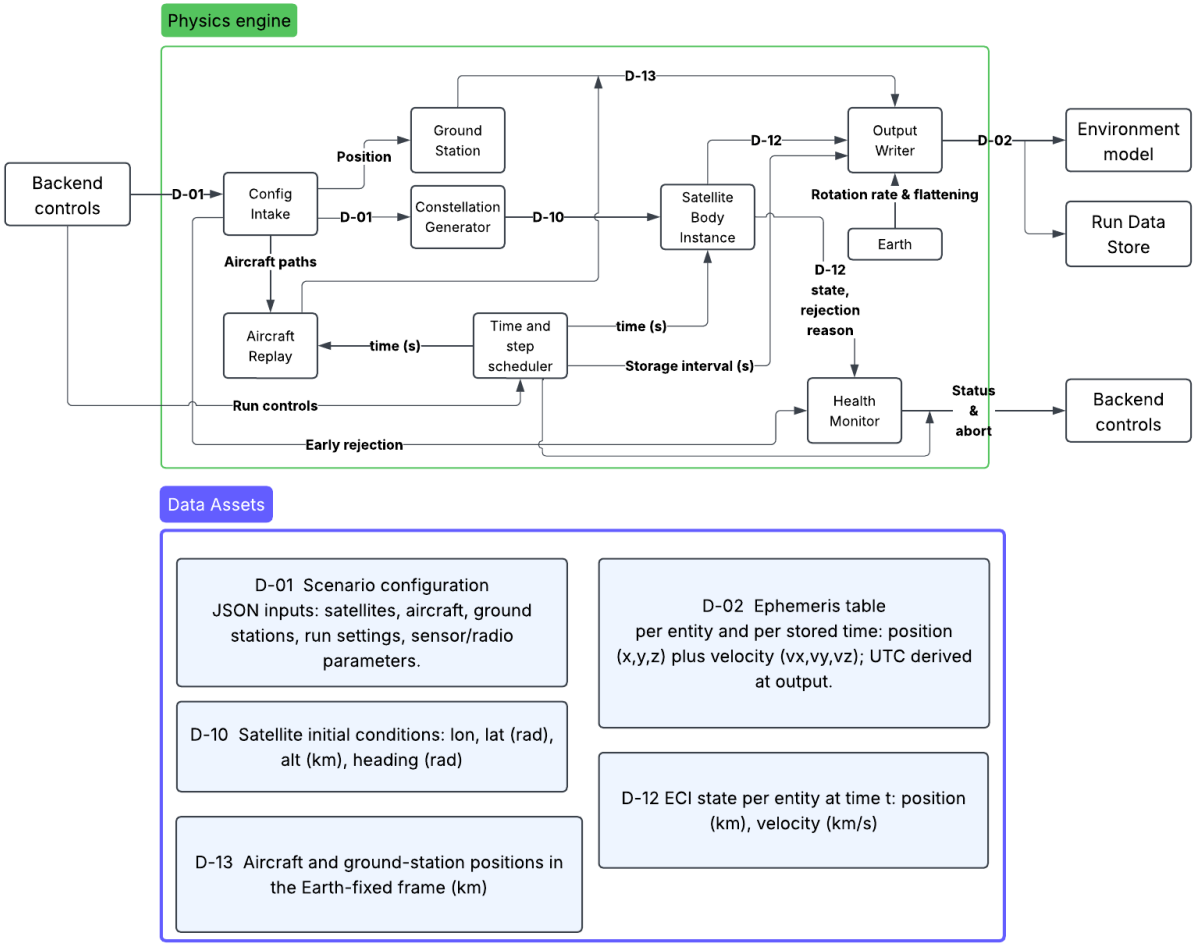


Figure 7: Physics Engine FBD

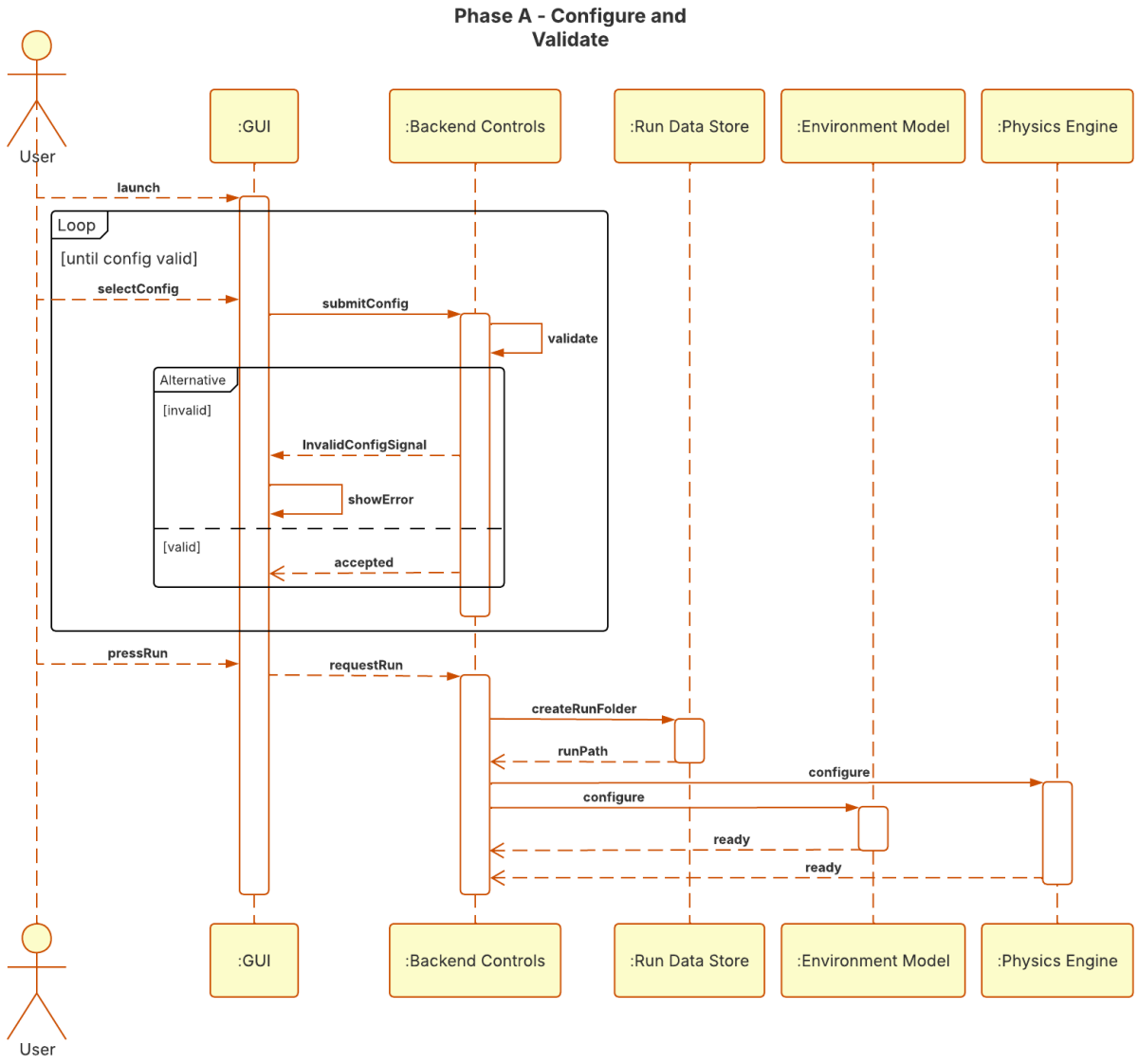


Figure 8: Sequence diagram for scenario configuration and validation

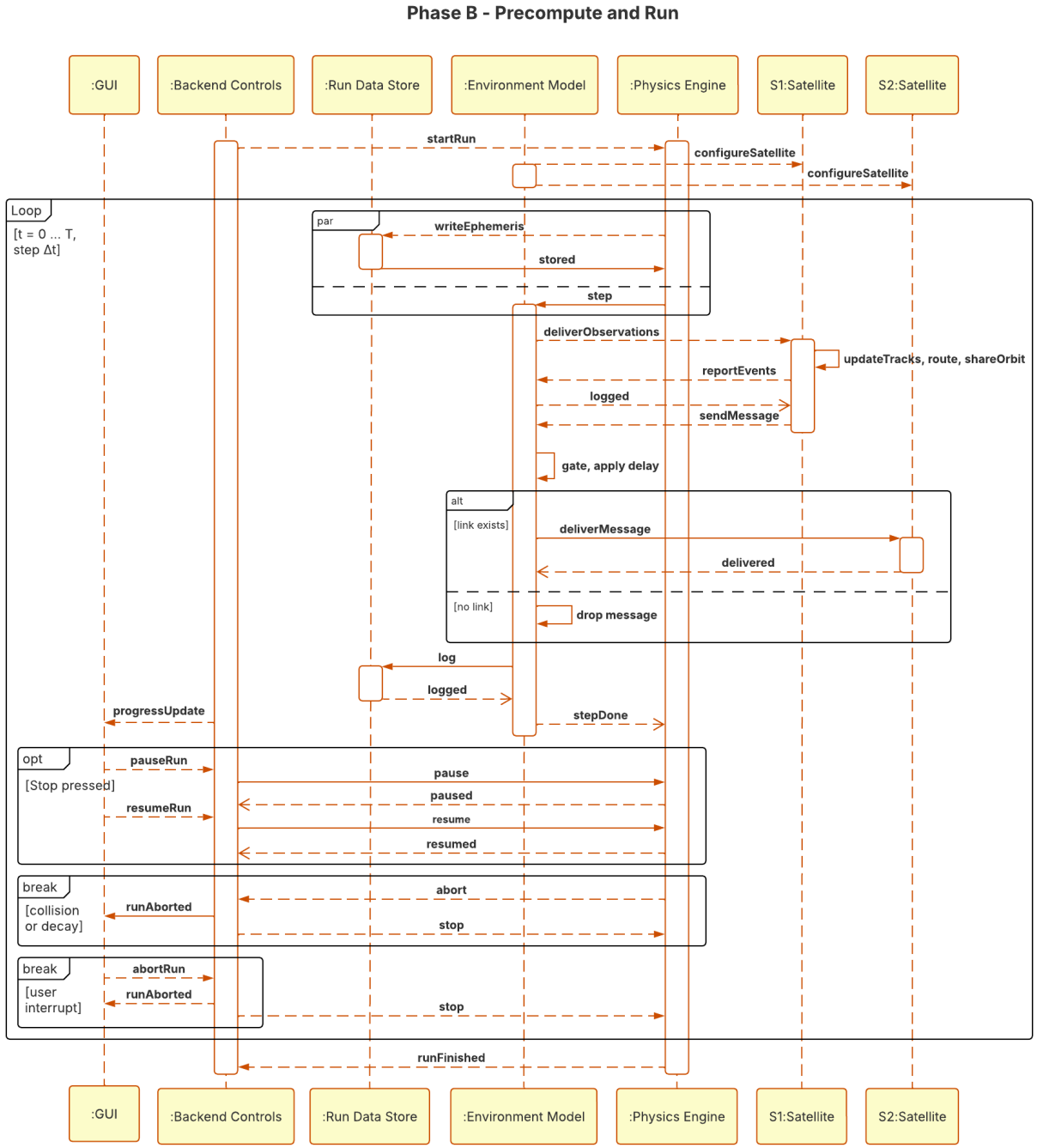


Figure 9: Sequence diagram for scenario pre-computing simulation run

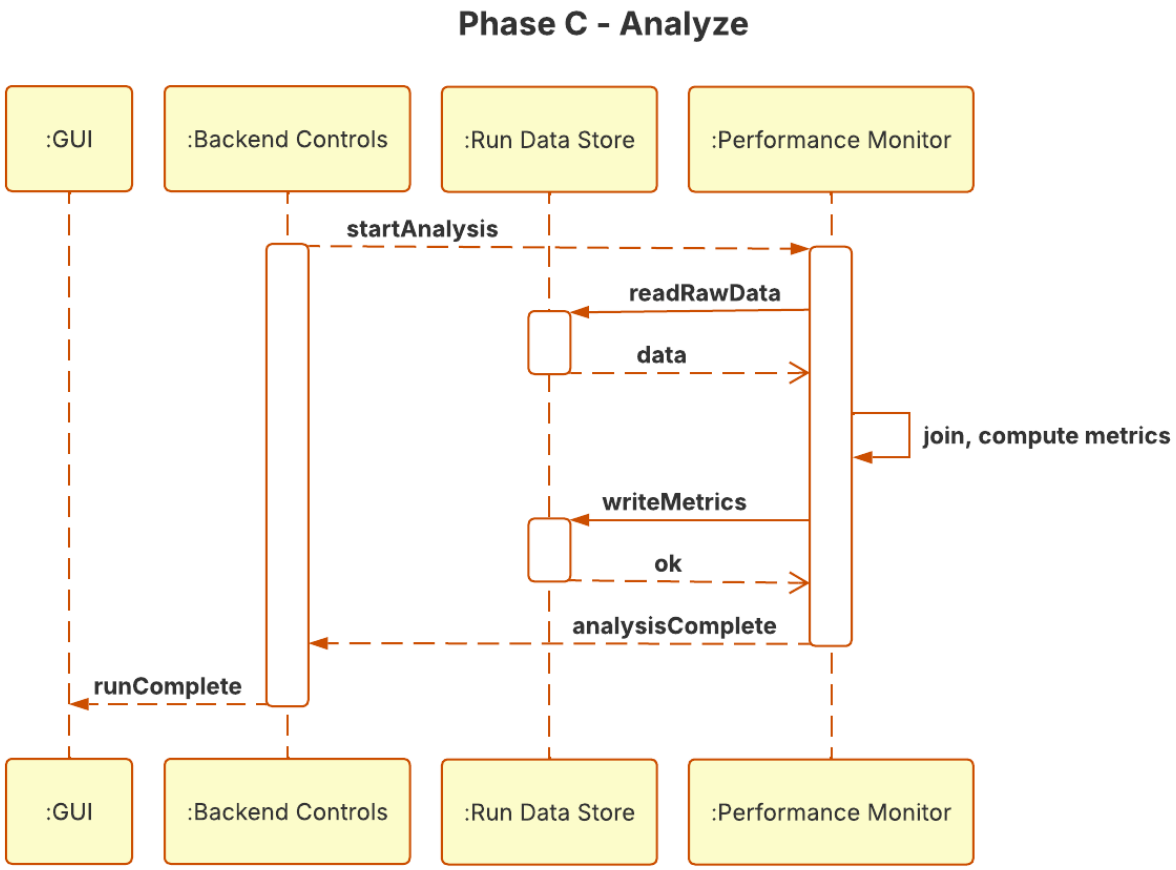


Figure 10: Sequence diagram for analyzing simulation run

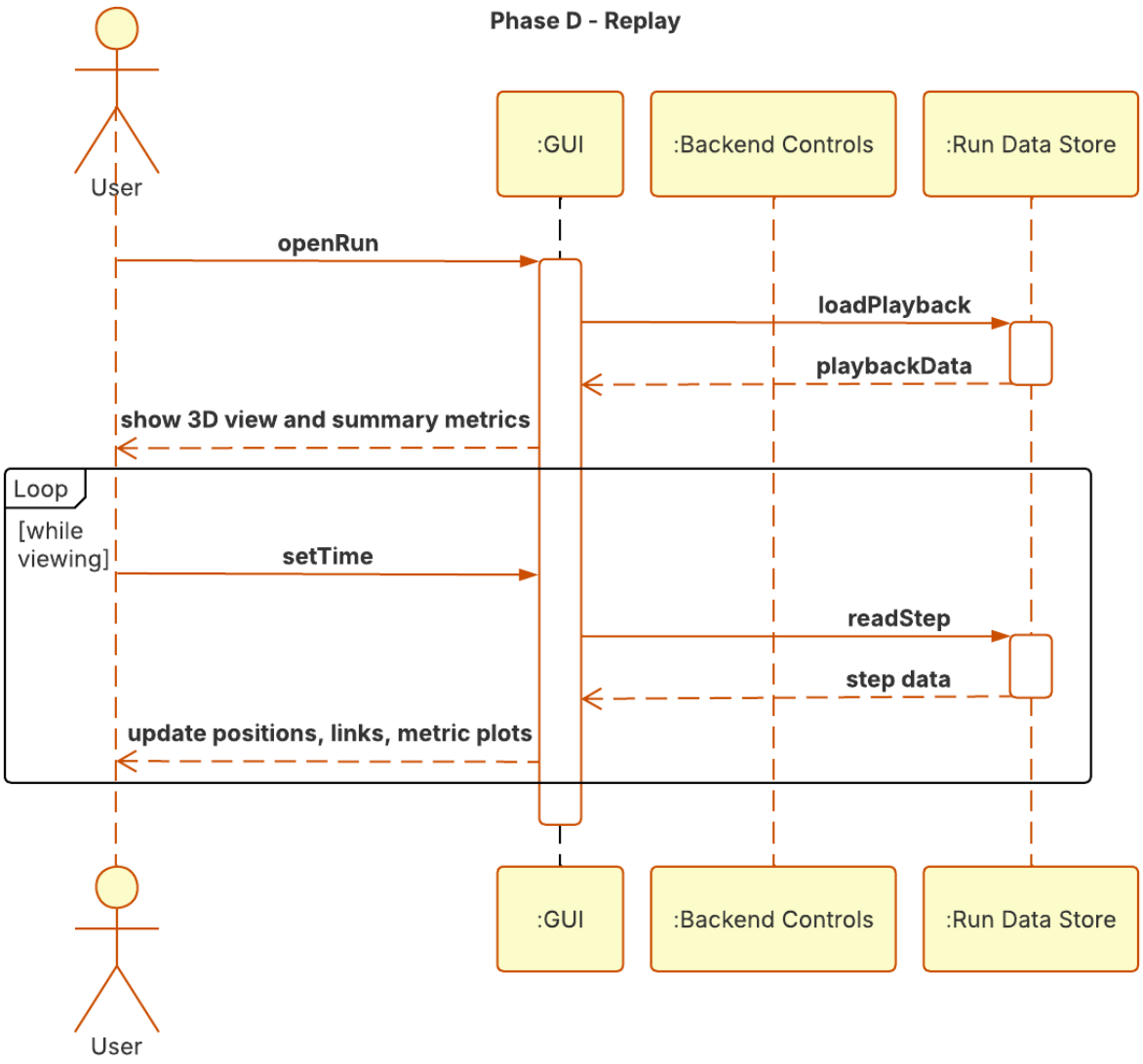


Figure 11: Sequence diagram for user playback of simulation run

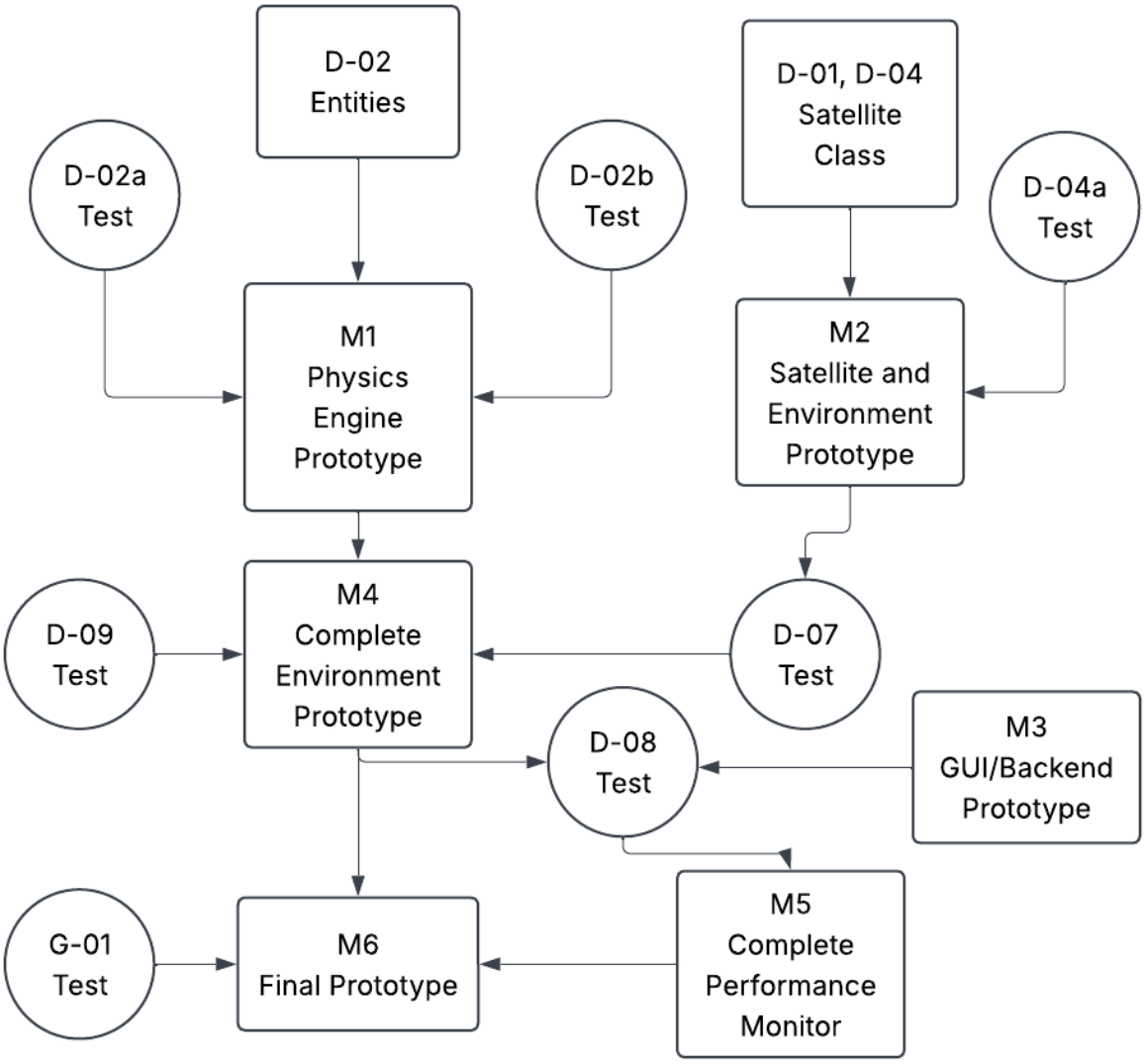


Figure 12: Full testing and prototype process diagram